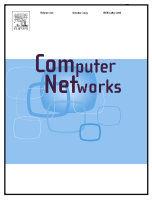
Computer Networks 273 (2025) 111764



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





PrivHFL: A privacy-preserving scheme for hierarchical federated learning

Bayan Alzahrani*, Dejun Yang

Computer Science, Colorado School of Mines, 1500 Illinois Street, Golden, CO, 80401, USA

ARTICLE INFO

Keywords:
Federated learning
Internet of Things (IoT)
Privacy protection
Homomorphic encryption

ABSTRACT

In the Internet of Things (IoT) field, where interconnected devices generate sensitive data, ensuring privacy is a major challenge. Federated Learning (FL) addresses this by allowing devices to collaboratively train a model without sharing their local data, improving privacy in IoT systems. While the traditional two-layer FL framework is commonly used, adopting a hierarchical client-edge-cloud architecture can significantly accelerate model training, especially in resource-constrained IoT networks. Hierarchical Federated Learning (HFL) offers significant advantages, yet concerns persist regarding potential privacy breaches from analyzing client or edge server data. To address these concerns, we propose PrivHFL, a privacy-preserving solution for HFL that leverages threshold homomorphic encryption. Security and performance analyses demonstrate that the proposed scheme is scalable, supporting larger FL scenarios, including diverse IoT environments, while ensuring data privacy. PrivHFL is resilient to collusion among nearly half of the clients and effectively handles client dropouts. Our approach achieves high accuracy in IID and non-IID scenarios, as demonstrated using the MNIST, CIFAR-10, and CIFAR-100 datasets. Additionally, we show that the added encryption overhead is reasonable, making our solution feasible for real-world IoT applications.

1. Introduction

The rapid proliferation of Internet of Things (IoT) devices, along with smartphones, wearable sensors, and autonomous vehicles, generates massive amounts of data daily. Utilizing machine learning (ML) to analyze these extensive datasets facilitates the generation of insightful predictions and interpretations. Traditionally, ML solutions have processed this data centrally, relying on robust models trained on a central server with the collected data. However, this approach raises significant privacy concerns due to handling sensitive personal information.

To mitigate these privacy issues, Federated Learning (FL) was introduced [1,2], which supports the development of collaborative models without sharing local data among devices. In FL, selected devices update model parameters using their local datasets. These updates are subsequently aggregated on a parameter server, such as a cloud or edge server, enhancing privacy in sensitive applications like medical image analysis [3,4].

Despite its advantages, FL, especially in its traditional client-server configuration, faces challenges such as potential server bottlenecks [5–7]. With the emergence of edge computing, hierarchical federal learning (HFL) has become a viable solution. This structure incorporates three layers–clients, edge servers, and a cloud server–to balance centralized and decentralized systems' benefits, addressing scalability, robustness, and latency issues [5,7]. Compared to the traditional two-layer FL architecture (client-edge or client-cloud), the client-edge-cloud HFL leverages edge computing to significantly reduce communication overhead between the cloud server and users, thus accelerating the training process [8]. Furthermore, HFL employs the cloud to connect with more users, enhancing overall training performance [8].

While FL and HFL significantly enhance client privacy, concerns about security and the potential for privacy leaks from model parameters persist [9–11]. Advanced technologies such as Homomorphic Encryption (HE) [12], Differential Privacy (DP) [13], and Secure Multiparty Computation (SMC) [14] have been proposed to safeguard the confidentiality of model parameters within FL and HFL frameworks. These technologies are crucial for providing privacy-preserving mechanisms that handle and exchange sensitive model information.

However, each technology has drawbacks; HE can increase computational overhead as computations are performed on encrypted data [15]. On the other hand, privacy-preserving mechanisms that rely on DP may compromise model accuracy due to noise addition [16]. MPC-based mechanisms employ several rounds of interaction among users to ensure that computations are carried out accurately and securely. However, they incur high communication costs and require participants to be online [17]. This raises an important question: Is it possible to

* Corresponding author.
E-mail address: balzahrani@mines.edu (B. Alzahrani).

https://doi.org/10.1016/j.comnet.2025.111764
Received 6 March 2025; Received in revised form 19 August 2025; Accepted 5 October 2025
Available online 9 October 2025
1389-1286/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
A comparative summary of existing FL schemes.

Scheme	Model	No Trusted Party	Honest-but-curious Server(s)	Honest-but-curious Users	Colluding Users	Post-quantum Security	No Decrease in Accuracy	Comm. Rounds	Robust to Dropout
Phong et al. [21]	Central server	✓	✓	✗	✗	✗	✓	1	✓
Zhang et al. [22]	Central server with KGC	✓	✓	✗	✗	✗	✓	1	✓
Ma et al. [26]	Central server with KGC	✓	✓	✓	✓	✓	✓	2	✗
Du et al. [18]	Central server with PKG	✓	✓	✓	✓	✓	✓	2	✓
Bonawitz et al. [17]	Central server	✓	✓	✓	✓	✗	✓	3	✓
Bell et al. [27]	Central server	✓	✓	✓	✗	✗	✓	3	✓
Sotthiwat et al. [28]	Central server	✓	✓	✗	✗	✗	✓	1	✗
Geyer et al. [29]	Central server	✓	✗	✓	✗	✗	✗	1	✓
Wei et al. [30]	Central server	✓	✓	✓	✓	✗	✗	1	✓
Truex et al. [31]	Central server with KGC	✓	✓	✓	✓	✗	✗	2	✓
HybridAlpha [32]	Central server with TTP	✗	✓	✓	✓	✗	✗	1	✓
HierFAVG [8]	Hierarchical FL	✓	✗	✗	✗	✗	✓	1	✓
HFL-DP [34]	Hierarchical FL	✓	✓	✓	✗	✗	✗	1	✓
PrivHFL (this work)	Hierarchical FL with KGC	✓	✓	✓	✓	✓	✓	1	✓

develop an HFL model that maintains user privacy while achieving high accuracy with manageable computational costs?

This study presents a novel privacy-preserving hierarchical federated learning PrivHFL framework tailored for environments like IoT networks. By integrating HFL with the HE system [18], PrivHFL enhances data privacy in distributed IoT systems. IoT devices, which generate vast amounts of sensitive data but are often constrained by limited computational and communication capabilities, are well-suited for HFL. Our approach leverages the hierarchical client-edge-cloud architecture [8] to accelerate model training and optimize learning efficiency in IoT networks. Edge servers, strategically placed closer to IoT devices, reduce communication overhead and latency, which is essential for real-time or near-real-time IoT applications. Furthermore, by grouping IoT devices into smaller clusters, we enable parallel computation, significantly speeding up the training process. On the other hand, the HE system ensures secure aggregation of user models, maintaining data privacy throughout the learning process. The following are the main contributions of this paper:

- We propose PrivHFL, a groundbreaking solution that addresses the privacy challenges in the widely adopted client-edge-cloud hierarchical federated learning architecture [8]. Our approach integrates a lightweight yet highly effective privacy-preserving mechanism using a multi-key HE method, tMK-CKKS [18], to ensure data protection against semi-honest IoT devices, edges, and cloud servers.
- We introduce a robust verification mechanism at the edge level to thwart potential collusion attacks among IoT devices within the same group, ensuring secure data aggregation and processing.
- Security and performance evaluations show that PrivHFL is highly scalable, effectively supporting large-scale FL scenarios with diverse IoT devices while maintaining robust data security. Additionally, PrivHFL demonstrates resilience against collusion among nearly half of the IoT devices and effectively manages device dropouts.
- We evaluate PrivHFL on real-world datasets, demonstrating that our model maintains high accuracy across independent and identically distributed (IID) and non-IID data settings, while keeping computational costs reasonable, which is critical for resource-constrained IoT environments.

The rest of the paper is organized in the following manner: Section 2 discusses related work. Section 3 outlines the preliminaries, and Section 4 describes the system and threat models along with the design goals. Section 5 presents the proposed PrivHFL scheme, and Section 6 provides a theoretical analysis. Section 7 evaluates PrivHFL's performance, and Section 8 concludes the paper.

2. Related work

This section reviews the literature on existing privacy-preserving federated learning schemes (PPFL) and HFL. Table 1 compares PrivHFL with existing FL and HFL schemes based on different features.

2.1. Privacy-preserving federated learning

FL schemes use various techniques to enhance privacy, such as HE [12], SMC [14], DP [13], and hybrid PPFL methods.

2.1.1. Homomorphic encryption

HE [12] is a cryptographic method that preserves data privacy by enabling calculations on encrypted data without decryption. Different types of HE schemes offer distinct capabilities and trade-offs. Partially Homomorphic Encryption (PHE) [19] allows only one type of operation–either addition or multiplication–on encrypted data. In contrast, Fully Homomorphic Encryption (FHE) [20] enables both addition and multiplication to be performed on encrypted data. Between PHE and FHE is Somewhat homomorphic encryption (SHE) allows for a limited number of additions and multiplications before losing correctness.

Phong et al. [21] proposed utilizing the Paillier encryption scheme [19] to secure model updates in the training process. Under this approach, users compute the model gradient using their dataset, encrypt the result, and transfer it to the server to update the encrypted global model parameters. Nevertheless, HE in their study results in substantial computational overhead. To design an efficient protocol, Zhang et al. [22] presented a PPFL approach that uses the Paillier encryption scheme [19] to encrypt the user's local gradients. The server performs secure aggregation with the encrypted gradients. This approach offers the advantage of reducing computation costs using a Distributed Selective Stochastic Gradient Descent (DSSGD) [23] technique in the local training step. DSSGD is an iterative optimization algorithm used to train the ML model on a large dataset, where users choose a subset of

parameters that significantly impact updating the gradients at each iteration. However, using the Paillier scheme for encryption involves costly modular exponentiation, where the message serves as the exponent. As a result, the communication and computational costs involved in this process remain substantial.

Unlike traditional PHE schemes, advanced FHE schemes have demonstrated significant efficiency improvements [24]. Although several FHE schemes are available, this paper focuses explicitly on the MK-CKKS scheme [25], which notably reduces computational and communication overhead through a ciphertext packing technique. Ma et al. [26] proposed a notable advancement in integrating MK-CKKS with FL, known as xMK-CKKS. The xMK-CKKS scheme enhances the MK-CKKS method [25] by enabling the server to decrypt only the aggregated encrypted model updates, ensuring the privacy of user updates. However, the xMK-CKKS scheme requires the participation of all clients in decrypting the aggregated model, making it unsuitable for scenarios where users may drop out during the learning process. To address this limitation, Du et al. [18] proposed a threshold variant of the MK-CKKS scheme (tMK-CKKS), which requires only a subset of clients for decryption, thus tolerating potential client dropouts. Nevertheless, their system requires two rounds of communication in each iteration, which may affect its efficiency.

2.1.2. Secure multiparty computation

SMC [14] is a cryptographic protocol that allows multiple parties to jointly perform a computational task without exposing their private data. Bonawitz et al. [17] proposed an FL scheme incorporating a secure aggregation protocol based on SMC to preserve the privacy of user model updates. In this method, the server is unable to access individual updates but can view the aggregated global model. This approach offers high privacy protection and maintains the original accuracy of the model. However, the aggregation process involves at least three communication rounds between users and the server in a single iteration, which results in significant communication overhead for clients with limited resources.

Several methods have been suggested to enhance SMC efficiency. Bell et al. [27] introduced a secure method for data aggregation that has a polylogarithmic computation and communication overhead. Their innovative approach involves shifting from a star topology used in the Bonawitz framework [17] to random subsets of users and applying secret sharing selectively among subsets of clients rather than across all pairs. Their approach significantly reduces overhead compared to the Bonawitz method [17], though it still requires three rounds of communication between the server and clients per iteration. Sotthiwat et al. [28] introduced a strategy that employs SMC to encrypt only the first layer of the local models, sending the remaining layers directly to the centralized server. This approach effectively minimizes the additional computational and communication costs associated with full SMC while retaining SMC's benefits in terms of privacy preservation and model accuracy. However, this protocol requires synchronous updates from all participants for each training iteration and lacks tolerance for participants failing to complete the protocol.

2.1.3. Differential privacy

DP [13] is a way to protect the privacy of individuals in datasets by adding noise to the data. This technique aims to ensure that including or excluding a single data point has a negligible impact on the outcome of a model. However, the inclusion of noise in differential privacy might result in a reduction in accuracy. Geyer et al. [29] introduced an FL framework incorporating differential privacy to protect user updates. The server randomly selects participants who update their local models and share their weights. These weights are then aggregated into a global model with the addition of random Gaussian noise, preventing malicious users from extracting information about others from the model. Nevertheless, this scheme is susceptible to malicious servers, as they can access unprotected updated parameters from users.

To address this vulnerability, Wei et al. [30] proposed a method that adds noise to the local gradients instead of the aggregation update. This makes it more difficult for malicious servers to infer user dataset information. In essence, when compared to FL methods based on aggregated model differential privacy, adopting user-level differential privacy in FL methods offers a more robust privacy guarantee as it protects against malicious users and servers. Nevertheless, more iterations or users can increase noise in this method, negatively impacting model performance.

2.1.4. Hybrid PPFL methods

Recently, hybrid PPFL approaches have been proposed to address the trade-offs between data privacy and model performance. The method proposed in this paper [31] combines differential privacy and SMC to enhance privacy protection within the FL framework without compromising model accuracy. By utilizing SMC, the proposed method can reduce the amount of noise added while protecting the model's privacy. This method provides the advantage of balancing confidentiality and performance without compromising either. Nevertheless, their system needs three rounds of communication in each iteration, making it unrealistic for FL implementations in real-world scenarios.

Xu et al. [32] introduced HybridAlpha, a novel method that integrates functional encryption with differential privacy to ensure data privacy in FL. In this system, a trusted third party generates a public key for each user from a master key pair, which is then used to encrypt the user's model parameters. The server requests the function private key from the trusted third party to aggregate the model updates based on the responses received. This approach ensures that the server obtains only the function results, which is the average weight, without accessing the individual weights of the users' local models. HybridAlpha significantly reduces the communication cost compared to the previous method [31]. However, their system requires a trusted third party to maintain the master private/public key pair and monitor the server to prevent manipulation of the aggregation weights.

2.2. Hierarchical federated learning

As the number of distributed devices increases, high latency and communication overhead become significant challenges for FL [33]. HFL addresses these issues. Liu et al. [8] proposed a 3-2 model with three layers–cloud server, edge servers, and end devices–and two levels of aggregation: local aggregation by end devices and edge servers, and global aggregation by edge servers and the cloud server. This method reduces communication demands by leveraging the proximity of edge servers and improves access efficiency compared to a remote cloud. HFL achieves higher model performance than edge-based FL as the cloud server can access more data. However, like traditional FL, HFL risks privacy leakage from inference attacks. To enhance privacy, Shi et al. [34] designed an HFL scheme based on Local Differential Privacy (LDP), adding noise to model parameters before sending them to servers. While this improves privacy, it may reduce accuracy, especially with more participants and global iterations, and its effectiveness with non-IID data remains unclear.

3. Preliminaries

This section provides an overview of the background and describes the foundational components of our proposed framework.

3.1. Federated learning

FL [1] is a decentralized ML framework that keeps training data localized on clients while the server stores a joint model, ensuring data remains distributed across different devices, often in a non-IID manner [35]. This paper focuses on supervised learning, where each user holds a private dataset. The dataset for user U_x consists of input samples and

corresponding class labels. FL models use parameters (weights and biases) to optimize the cost function, which measures the difference between predicted and actual values, aiming to minimize the loss function $F(w)$ across the global dataset. The FedAvg algorithm involves initializing global model parameters on the server and users performing local updates by computing gradients and updating local parameters. Then the server aggregates these updates to refine the global model. This process iterates until the model achieves the desired accuracy or completes a set number of iterations. In FedAvg, only model parameters are communicated, thus avoiding transmitting raw data [1]. In our scheme, we enhance data privacy by encrypting the local model parameters using the public key pk and performing the aggregation on the encrypted parameters. The following subsections describe the learning processes of two FL types: the traditional and hierarchical FL systems.

3.1.1. Two-layer FL

FL typically involves m users and either an edge or cloud server. The cloud-based FL can access more data but often requires significant communication with high latency [8]. In contrast, the edge-based FL can communicate more efficiently with a limited number of devices, reducing training performance [8].

3.1.2. Hierarchical FL

In an HFL system [8], three entities are distributed across three logistical layers. The cloud server, which serves as the root, is connected to multiple N edge servers denoted by i in the middle layer. Each edge server is linked to a distinct set of users, $\{U^i\}_{i=1}^{N}$. The HFL consists of m users indexed by x and i $\{U_x^i\}_{x=1}^{j}$, and these entities are organized in a tree structure. Local model updates from users are transmitted to their respective edge servers, which aggregate these updates. Several rounds of aggregation occur between each edge server and its users before the model updates are sent to the cloud server. The aggregation process then extends from the edge layer to the cloud server. After cloud aggregation, the global model parameters are traversed down the hierarchy to reach the users.

In contrast to the two-layer FL setup, HFL incorporates edge and cloud servers. HFL can access vast training data using a cloud server, while each edge server can quickly update its model based on inputs from local users. By positioning edge servers closer to users, HFL minimizes the transmission delay and significantly reduces communication costs [8]. In the PrivHFL scheme, we have developed a privacy-preserving scheme based on the HierFAVG algorithm [8].

3.2. Threshold multi-key homomorphic encryption

The threshold multi-key homomorphic encryption (tMK-HE) [18] is a variant of the MK-CKKS scheme [25], which is based on the RLWE HE scheme [36]. In the tMK-CKKS, the master secret key (sk) is divided into n shares using Shamir's secret sharing scheme [37], ensuring that less than t shares of a secret key cannot reconstruct the original sk. In our scheme, we employ a distributed decryption approach that requires the collaboration of at least two shares to decrypt ciphertext.

Our scheme does not use homomorphic multiplication; thus, we focus solely on introducing the concept of homomorphic addition in the tMK-CKKS. The tMK-CKKS includes the following algorithms:

- Setup$(1^\epsilon) \rightarrow (n, q, \gamma, \psi, \phi, a)$: Let ε be the security parameter, define the RLWE dimension n, ciphertext modulus q, γ for key distribution, and ψ and ϕ for error distributions. Generate a random shared polynomial a.
- KeyGen$(n, q, \gamma, \psi, \phi, a) \rightarrow (mp)$: Encode a plaintext m as a polynomial of ring R.
- ENC$(mp, b, a) \rightarrow ct_i$: Encrypt encoded plaintext mp such that:

$$ct_i = (c_0, c_1) = (v.b + mp + e_0, v.a + e_1) \bmod q, \quad (1)$$

where $a = a[0], b = b[0], v \rightarrow \gamma$, and $e_0, e_1 \rightarrow \psi$.

- Agg$(ct_i^x) \rightarrow (C_{sum})$: The sum of the ciphertexts is computed as follows:

$$C_{sum} = \sum_{x=1}^{j} ct_i = (c_{sum_0}, c_{sum_1}) = \left(\sum_{x=1}^{j} ct_0^x, \sum_{x=1}^{j} ct_1^x\right), \\ = \left(\sum_{x=1}^{j} (v^x.b + mp^x + e_0^x), \sum_{x=1}^{j} (v^x.a + e_1^x)\right) \\ \bmod q. \quad (2)$$

- PartDec$(c_{sum_1}, s_i) \rightarrow d_i$: Given a ciphertext c_{sum_1}, a share of secret key s_i, and an error e_i^* drawn from ϕ, the partially decrypted share d_i can be obtained by computing:

$$d_i = s_j.c_{sum_1} + e_i^*, \quad (3)$$

where $s_j = s_i \prod_{l \in t \setminus i} (i/i - l)$.

- FullDec$(c_{sum_0}, d_t) \rightarrow mp$: After receiving t partially decrypted shares of ciphertexts, the plaintext mp can be obtained by computing:

$$mp = c_{sum_0} + \sum_{i=1}^{t} (d_i) \bmod q. \quad (4)$$

- Dcd$(mp) \rightarrow (m)$: recover the plaintext m from the polynomial mp.

Decrypting an encrypted number in the tMK-CKKS [18] requires using both the PartDec and FullDec algorithms. tMK-CKKS ensures security against non-colluding semi-honest adversaries [36], preventing potential information leakage from ciphertexts.

4. Problem definition

This section presents the PrivHFL system model, describes the attack model, and defines design goals.

4.1. System model

As illustrated in Fig. 1, our scheme comprises four fundamental entities:

- Key Generation Center (KGC): Responsible for generating and distributing system keys, including public and secret key shares for cloud servers, edge servers, and users. The KGC's involvement is limited to the initial phase.

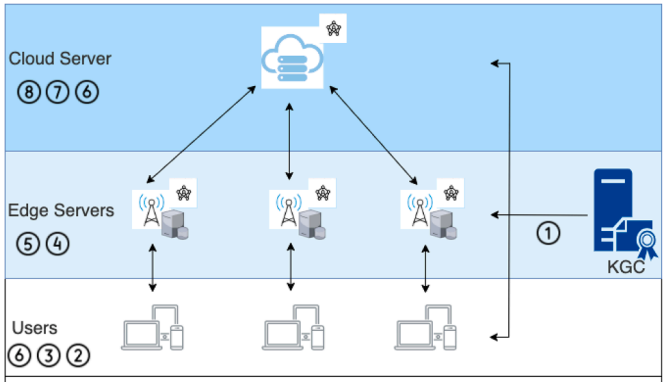


Fig. 1. An overview of our proposed PrivHFL scheme.

- Users (U_x^i): In each training iteration, users conduct local training on their datasets D_x^i and encrypt their local model parameters $w_x^i(k)$ with the public key before uploading them to the corresponding edge server E_i. Edge servers then independently aggregate the models from associated users, and the cloud server aggregates models from all edge servers after a set number of aggregations.
- Edge Servers (E_i): Serve as mediators between users and the cloud server, performing multiple model parameter aggregations to accelerate training. Each edge server connects to a set of users.
- Cloud Server (CS): Acts as a central coordinator, aggregating model parameters uploaded by edge servers and facilitating data sharing among users.

4.2. Threat model

We assume that users, edge servers, and the cloud server operate as honest but curious entities. This means they will faithfully follow the designated protocol but are curious about users' model updates. These entities are expected not to collude with each other, meaning they will not share additional information during the training process.

Additionally, we consider the possibility of collusion among users within the same group to gain other users' private information. However, our assumption in this work is $CU < \lceil \frac{j}{2} \rceil$, where CU represents the number of colluding users and j is the total number of users in that group.

4.3. Design goals

Considering the threat model mentioned above, the system has two design goals:

- Privacy: PrivHFL should ensure the confidentiality of user information, preventing any leakage to honest but curious entities under both IID and non-IID data settings.
- Accuracy: We aim to maintain the accuracy of the global model while ensuring privacy. The accuracy of the global model should be as similar as possible to that of the HierFAVG algorithm [8].
- Efficiency: The running time should be minimized as much as possible, and the number of communication rounds should be comparable to state-of-the-art methods.

5. Proposed PrivHFL scheme

In this section, we first give an overview of the PrivHFL scheme and then discuss the details of our proposed scheme.

5.1. System overview

To achieve our design objectives, we propose a privacy-preserving hierarchical federated learning scheme that involves multiple users collaborating to train a global model managed by untrusted edge and cloud servers. Specifically, users train the global model on their private datasets and transmit the model parameters to their corresponding edge after multiple local updates. Each edge aggregates its users' models and, after multiple local aggregations, forwards the aggregated model to the cloud server. It is assumed that the curious users, edges, and cloud server might attempt to infer private information about users' training datasets using the received parameters. Fig. 1 provides a high-level overview of our proposed PrivHFL scheme.

To ensure privacy, we use the threshold multi-key HE scheme [18] to encrypt each user's model parameters, preventing the curious users, edges, and cloud server from determining the actual values of the users' updates. In this manner, the edges can only aggregate the ciphertexts according to the FL protocol. In contrast, users have access only to the aggregated model, thus preserving the confidentiality of individual model parameters. Furthermore, the cloud server receives model aggregations from the edges only after multiple rounds, thereby providing a robust privacy guarantee for users' private training data.

Algorithm 1 Setup.

Input: Security parameter ε.
Output: System keys: public key b, secret shares $s_{E_i}^U$, $s_{E_i}^E$, $s_{E_i}^{CS}$.
// **KGC - Key Distribution**
$(n, q, \gamma, \psi, \phi, a) \leftarrow \text{Setup}(1^\epsilon)$. ▷ KGC generates RLWE parameters.
for each $E_i \in [1, N]$ in E **do**
  $(b, s_{E_i}^U, s_{E_i}^E, s_{E_i}^{CS}) \leftarrow \text{KeyGen}(n, q, \gamma, \psi, \phi, a)$. ▷ KGC generates user, edge, and cloud shares.
end for
return KGC broadcasts b and distributes $s_{E_i}^U$ to to all users under edge E_i, $s_{E_i}^E$ to edge E_i, and $s_{E_i}^{CS}$ to cloud server.
// **Cloud Server - Model Distribution**
CS initializes all users with the model parameters $w(0)$.

5.2. Design of PrivHFL

PrivHFL consists of two main phases: 1) the setup phase, which involves key and model distribution, and 2) the privacy-preserving hierarchical federated learning phase, which includes local training, edge aggregation, and cloud aggregation. A detailed explanation of each phase follows.

5.2.1. Setup

During the setup phase, as outlined in Algorithm 1, KGC manages key distribution, while the CS distributes the initial model to all users via the edge servers.

a) Key distribution: The KGC sets up keys in the system for the cloud server, edge servers, and users. Typically, the KGC only participates in the initial setup phase and does not participate in subsequent processes. For each edge server E_i, KGC splits the master secret sk into three shares and then generates a specific secret key for each entity: $s_{E_i}^U$, $s_{E_i}^E$ and $s_{E_i}^{CS}$ which are sent to U_x^i, E_i and CS, respectively ($i \in [1, N]$ and $x \in [1, j]$). PrivHFL utilizes the KeyGen algorithm to divide and distribute the secret key among U_x^i, E_i, and CS to safeguard the confidentiality of the secret key.

The reliance on a single key cryptosystem could lead to a security vulnerability. If an adversary gains access to the secret key S, they could decrypt all information encrypted with that particular secret key, potentially compromising the privacy of the FL/HFL scheme [38]. To address potential privacy concerns, we implement the threshold multi-key HE scheme in the HFL scheme [18]. In this scheme, each entity (CS, E_i, U_x^i) holds a secret share of s_i. Two sequential algorithms, PartDec and FullDec, are employed to decrypt the ciphertext ct_i. PartDec is first used to generate the partial decryption share d_i using $s_{i\in[1,2,3]}$ as specified in Eq. (3). Subsequently, FullDec is executed using c_{sum_0}, d_t to recover the plaintext w, as outlined in Eq. (4). This approach ensures that even if the adversary acquires only one secret share, $s_{i\in[1,2,3]}$, the confidentiality of w remains secure.

b) Model distribution: CS sends the initial global model $w(0)$ to all users via their corresponding edge server E_i.

5.2.2. Privacy-preserving hierarchical federated learning (PPHFL)

As outlined in Algorithm 2, there are three primary processes: local training, edge aggregation, and cloud aggregation.

a) Local training: In the kth training iteration, each user U_x^i trains the model parameters locally ($w_x^i(k)$) and then normalizes the local model by multiplying it by the size of its dataset. Subsequently, each user employs the Ecd algorithm to convert the model parameters list into polynomial form and encrypt this polynomial before sending it to their corresponding edge server ($ct_i^x(k)$). It is important to note that after every k_1 local

Algorithm 2 Privacy-preserving hierarchical federated learning (PPHFL).

for $k = 1, 2, \ldots, K$ **do** ▷ K: total number of training iterations.
  // Phase 1: User Local Training
  for each user $x = 1, 2, \ldots, m$ in parallel **do**
    $w_x^i(k) \leftarrow w_x^i(k-1) - \eta \nabla F_x(w_x^i(k-1))$ ▷ Update local model for user x at edge i.
    if $k|\kappa_1 = 0$ **then** ▷ Send encrypted updates every κ_1 rounds.
      Encode and encrypt $w_x^i(k)$ with the master public key (b, a) and send $(ct_i^x(k))$ to corresponding edge server
    end if
  end for
  // Phase 2: Edge Aggregation
  for each edge $i = 1, \ldots, N$ in parallel **do**
    if $|(ct_i^x(k))| \geq \left\lceil \frac{j}{2} \right\rceil + 1$ **then** ▷ minimum ciphertexts required to prevent collusion attacks.
      $c_{sum_0}(k), c_{sum_1}(k) \leftarrow \text{EdgeAgg}(ct_i^x(k))$ ▷ Aggregate encrypted models.
      $[d_1(k)] \leftarrow \text{PartDec}(c_{sum_1}(k))$ using $s_{E_i}^E$ ▷ Edge partial decryption.
    end if
    if $k|(\kappa_1\kappa_2) \neq 0$ **then**
      for each user $x \in U^i$ in parallel **do**
        $[d_2(k)] \leftarrow \text{PartDec}(c_{sum_1}(k))$ using $s_{E_i}^U$ ▷ User partial decryption.
        $w^i(k) \leftarrow \text{FullDec}(c_{sum_0}(k), d_1(k), d_2(k))$ ▷ Recover aggregated model.
        $w_x^i(k) \leftarrow \frac{w^i(k)}{|D^i|}$
      end for
    else
      // Phase 3: Cloud aggregation:
      for each update from E_i **do**
        $[d_2(k)] \leftarrow \text{PartDec}(c_{sum_1}(k))$ using $s_{E_i}^{CS}$ ▷ Cloud partial decryption
        $w^i(k) \leftarrow \text{FullDec}(c_{sum_0}(k), d_1(k), d_2(k))$ ▷ Recover aggregated model.
      end for
      $w(k) \leftarrow \text{CloudAgg}(\{w^i(k)\}_{i=1}^N)$ ▷ Compute global model from N edge updates.
      //Broadcast updated global model to all users.
      for each user $x = 1, \ldots, m$ in parallel **do**
        $w_x^i(k) \leftarrow w(k)$ ▷ Update local model for user x at edge i with global model.
      end for
    end if
  end for
end for

updates, each user sends the encrypted model parameters to their edge server.

Since HE operates with integers, we utilize an encoding method to convert floating-point numbers to integers with a specified degree of precision before encryption. For $z \in [1, n]$, each entry w_{xz}^i of the model parameters should be converted into integer form as follows:

$$w_x^i \leftarrow \{\lfloor \deg . w_{xz}^i \rceil\}_{z=1}^n. \tag{5}$$

Let $\lfloor r \rceil$ denote the nearest integer to the real number r, deg represents the precision degree, and w_{xz}^i corresponds to the zth entry in the model parameters w_x^i. In our scheme, we chose 10^8 as the value for deg to preserve accuracy. Previous work [39] has demonstrated that using such a degree of precision ensures that the accuracy is maintained.

b) Edge aggregation: Following every k_1 local update by each user, each edge server aggregates its users' encrypted weighted models:

$$\begin{aligned} C_{sum} &= \sum_{x=1}^{j} ct_i = \left(\sum_{x=1}^{j} ct_0^j, \sum_{x=1}^{j} ct_1^j \right), \\ &= (c_{sum_0}, c_{sum_1}) \bmod q. \end{aligned} \tag{6}$$

After aggregating the encrypted local model parameters c_{sum_0}, c_{sum_1}, the edge server partially decrypts the c_{sum_1} using the PartDec algorithm with the secret key share $s_{E_i}^E$. Subsequently, the edge server sends both the encrypted aggregated model C_{sum} and the partially decrypted aggregated model $[d_1(k)]$ back to the users. Users then apply the PartDec algorithm to c_{sum_1} with their own key $s_{E_i}^U$ to obtain $[d_2(k)]$. After obtaining two partially decrypted shares, each user uses the FullDec algorithm to decrypt the aggregated model and decode the plaintext polynomials to obtain $w^i(k)$. Finally, users average the aggregated model to derive the new global model.

Conversely, after every k_2 model aggregation at the edge server, the edge server partially decrypts the aggregated model $[d_1(k)]$, using PartDec with the secret key share $s_{E_i}^E$. Instead of sending the model aggregations back to the users, each edge server forwards the encrypted model aggregations C_{sum} and the partially decrypted aggregated model $[d_1(k)]$ to the cloud server.

c) Cloud aggregation: Following every k_2 model aggregation at each edge server, the cloud server uses PartDec and FullDec algorithms with the secret key share $s_{E_i}^{CS}$ to obtain $w^i(k)$. Then, the cloud server aggregates all the models from the edge servers:

$$w(k) \leftarrow \frac{\sum_{i=1}^{N} |D^i| w^i(k)}{|D|}. \tag{7}$$

Finally, the CS broadcasts $w(k)$ to all users U_x^i through edge servers E_i.

6. Theoretical analysis

In this section, we assess the security of the PrivHFL scheme and explore its computational and communication complexities.

6.1. Security analysis

We evaluate the security and privacy of the PrivHFL scheme by analyzing its resistance to various attacks, including those from honest but curious users, edge servers, and the cloud server. Additionally, we discuss how our scheme mitigates risks posed by colluding users.

6.1.1. Honest but curious security

Our scheme ensures that no information about user datasets is leaked to honest but curious users, edge servers, and the cloud server. The security of PrivHFL is closely tied to the security of the ring learning with errors (RLWE) [36], a cryptographic framework known for its robustness against quantum attacks. In the FL/HFL setting, user data remains confidential as it is solely utilized for local model training. Our scheme ensures that local model parameters are not transmitted in plaintext and that all data are encrypted during transmission using the tMK-CKKS scheme to safeguard user privacy. During training, each user transmits ciphertext ct_i^x to the edge server, which incorporates an added error to maintain security under the RLWE assumption [36]. RLWE ensures that ct_i^x is computationally indistinguishable from uniformly random elements of R, thereby preserving the confidentiality of the original plaintext from the edge server.

Moreover, the PrivHFL scheme guarantees that the edge servers can only perform certain operations over encrypted data. Consequently, the edge servers cannot decrypt the ciphertext without access to the master secret key. Although the edge servers possess a share of the secret key, they are incapable of reconstructing the master secret key to decrypt

the ciphertext, as the key is randomly divided through the KeyGen algorithm of the tMK-CKKS scheme [18]. Additionally, assuming that the three entities (users, edge servers, and cloud server) do not collude, no individual secret key share can reveal the master secret key S, as guaranteed by the Shamir secret sharing scheme [37]. Splitting the secret key into three shares ensures that fewer than two shares are insufficient to reconstruct the master secret key. Consequently, adversaries cannot recover the original plaintext with fewer than two shares of the partially decrypted ciphertexts.

On the other hand, the PrivHFL scheme ensures that entities performing full decryption, whether users or the cloud server, only have access to aggregated model parameters instead of the original data (i.e., the individual model parameters of each user).

6.1.2. Colluding users security

Before each edge aggregation, the edge server verifies that it has received responses from at least $\lceil \frac{j}{2} \rceil + 1$ of its users, given that more than half of the users in our scheme should not be colluding. This verification process helps prevent collusion attacks, as colluding users cannot isolate the response of a target user and extract private information from the output of the aggregation.

6.2. Complexity analysis

We compared the proposed PrivHFL against the original HFL [8], HFL with DP [34], and the threshold multi-key HE scheme tMK-CKKS [18] in terms of their communication and computational complexities. To ensure a fair comparison, we implemented the tMK-CKKS scheme [18] using the HFL framework to align it with the other baseline structures.

Let m represent the number of users, n the number of edge servers, s the model size, and x the number of parameters in the model. We evaluate the efficiency of the three schemes by analyzing the complexity involved in three key steps: 1) local training, 2) edge aggregation, and 3) cloud aggregation.

6.2.1. Communication overhead

The number of communication rounds remains consistent across all systems, except for the tMK-CKKS scheme [18], while the model sizes vary among them. The communication complexity of the local training step is dependent on the model size. In PrivHFL and tMK-CKKS, each user U_x^i encrypts their model parameters $(ct_i^x(k))$, so the ciphertext size of the model is approximately 88KB. In HierFAVG, the model size is 45KB, while in HFL-DP, it is 47KB.

The complexity of the edge aggregation step primarily depends on the number of users m, the model size s, and the number of communication rounds between users and the edge. In PrivHFL, the encrypted aggregated model size is 53KB. In HierFAVG, it is 45KB, and in HFL-DP, it is 48KB. In contrast, the tMK-CKKS scheme has a model size of 107KB, as it requires two communication rounds between users and the edge server to decrypt the aggregated ciphertext.

In the cloud aggregation step, the communication complexity depends on the number of edge servers n and the model size s. The model size is almost 44KB for PrivHFL, HierFAVG, and tMK-CKKS, while 48KB for HFL-DP.

6.2.2. Computation overhead

In the local training step, the computational complexity depends on the number of parameters in the model. To reduce the computation cost, our scheme can pack 2^{14} model parameters into a single plaintext and utilize the Single Instruction Multiple Data (SIMD) technique to perform encryption and other operations on these parameters in parallel [18,25].

The complexity of the edge aggregation step mainly depends on the number of users m and the number of parameters in the model x. The computation complexity in the cloud aggregation step depends on the number of edge servers n and the model parameters x.

6.3. Functionality

In this subsection, we compare the functional advantages of PrivHFL with other schemes, HierFAVG [8], HFL-DP [34], and tMK-CKKS [18]. Table 1 provides a summary of these comparisons.

In HierFAVG [8], users share local model parameters in plaintext form, leading to potential privacy leaks. To mitigate this, HFL-DP [34] employs DP to perturb the parameters before transmission. However, the HFL-DP [34] does not consider protection against colluding users. Our scheme's implementation of threshold HE and edge verification processes ensures robust data privacy against colluding users and maintains post-quantum security against honest but curious entities. Additionally, our scheme considers privacy preservation in both IID and non-IID data settings, whereas HFL-DP only conducts experiments with IID data. On the other hand, adding more users to the HFL-DP scheme tends to reduce its accuracy. In our scheme, we prioritize efficiency over accuracy. Reduced accuracy could make PPFL less attractive than the original FL, given that accuracy is a key consideration in the field. However, our scheme maintains the original FL accuracy. To improve efficiency, we can enhance the scheme by adding more processing units and optimizing the code.

When compared to the tMK-CKKS scheme [18], our system presents a significant advantage by utilizing a single key for all users within the same group, one key for their corresponding edge, and one for the cloud, requiring only two of these three keys for decryption. In contrast, tMK-CKKS assigns a unique key to each user which requires multiple keys during the decryption process. By reducing the number of keys involved, our approach simplifies key management, minimizes computational overhead during decryption, and enhances scalability and efficiency–especially in large-scale IoT environments–while maintaining robust privacy and security. Moreover, our system makes it more difficult for attackers, as they would need to compromise both an edge device and either an end device or the cloud to retrieve data. In contrast, with tMK-CKKS, an attacker would only need to compromise a set of end nodes, which typically have fewer security measures compared to edge devices and the cloud, especially in IoT environments. Importantly, our system maintains a high level of privacy while requiring fewer communication rounds compared to tMK-CKKS. This efficiency is achieved because PrivHFL eliminates the need to send encrypted aggregated model updates back to users for decryption.

7. Performance evaluation and discussions

In this section, we compare the performance of PrivHFL with the original HierFAVG [8], HFL-DP [34], and tMK-CKKS [18]. Moreover, we evaluate PrivHFL's performance in various settings, including: 1) different heterogeneous data settings, such as IID and non-IID, 2) performance across different levels of precision, and 3) with different numbers of users.

7.1. Experiment setup

All experiments were conducted using Python 3.0 on a Jupyter Server provided by Saturn Cloud, which has eight cores and 64 GB RAM. For the threshold multi-key HE, we adopted a 128-bit security level according to recommendations of the NIST and the HE security standard [40,41], with $n = 32{,}768$ bits, and ciphertext modulus size of $\log q \leq 881$ bits. The reported execution times are the average of five experiments.

1) Dataset: We evaluated our scheme on image classification tasks using standard datasets MNIST [42], CIFAR-10, and CIFAR-100 [43], with a Convolutional Neural Network (CNN).

2) Data Settings: We evaluated the PrivHFL scheme in both IID and non-IID settings:

- IID settings: Each edge server stores IID data, ensuring the training data is evenly distributed among all edges.

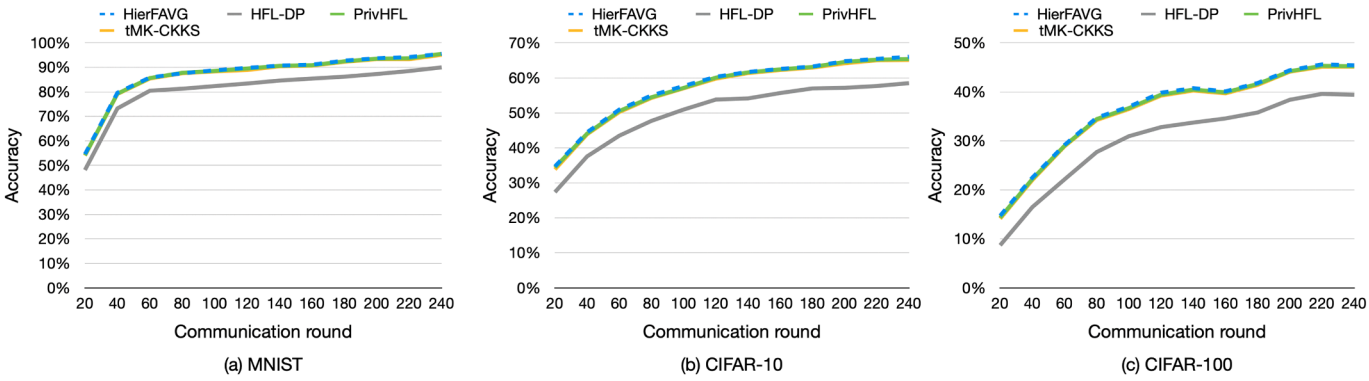


Fig. 2. Test accuracy on MNIST, CIFAR-10, and CIFAR-100 datasets over communication rounds in an IID setting.

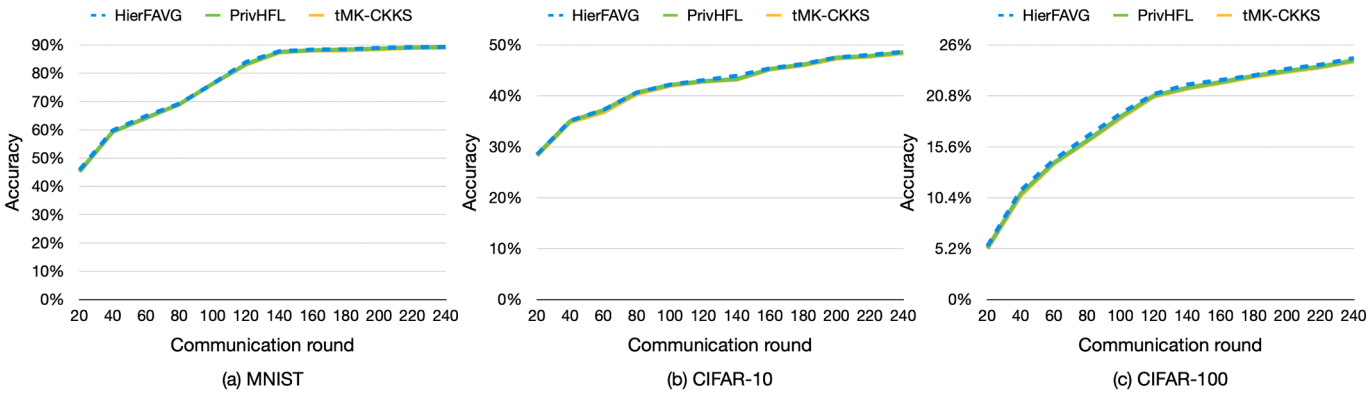


Fig. 3. Test accuracy on MNIST, CIFAR-10, and CIFAR-100 datasets over communication rounds in a non-IID setting.

- Non-IID settings: Each edge server stores non-IID data. The training data are categorized by class, with each edge holding samples from five distinct classes and each user possessing samples from only one class.

3) Model Architectures: We consider a client-edge-cloud HFL architecture comprising 50 users, five edge servers, and one cloud server. Each edge server is configured with an equal number of users and access to the same amount of training data. This paper uses multiple threads at the edge level to enhance efficiency further. Regarding other parameters, we set the number of local updates k_1 to 10 and the number of edge aggregation k_2 to 3 across different communication rounds K. We also set the precision degree to be $deg = 10^8$. Additionally, we used mini-batch Stochastic Gradient Descent (SGD) for local training with a batch size of 20 and a learning rate η of 0.01.

7.2. Experiment results

1) Comparative Evaluation: We compared our proposed method, PrivHFL, with three existing methods, HierFAVG [8], HFL-DP [34], and tMK-CKKS [18]. As shown in Fig. 2 for the IID setting, the accuracy of our method is comparable to that of the original HierFAVG method [8] and tMK-CKKS [18], while the difference in accuracy between our method and HFL-DP ranges from 5% to 8%. In the non-IID data setting, as shown in Fig. 3, the accuracy achieved by PrivHFL closely aligns with that of the original HierFAVG method [8] and tMK-CKKS [18]. It is important to note that we have not evaluated HFL-DP [34] in the non-IID scenario as this method does not support non-IID data.

2) The effect of parameters: Several factors influenced our method's accuracy and runtime, including the precision degree of the encoding process, the number of users, the number of local updates (k_1), and the number of edge aggregations (k_2). To evaluate our scheme, we conducted experiments by varying the precision degree and the number of users while keeping other factors constant.

- The effect of precision degrees: As depicted in Fig. 4(a), an increase in the precision degree enhances the accuracy of the decrypted global model, typically indicating that the test performance of the encrypted aggregated model closely matches that of the plaintext aggregated model. Nevertheless, Fig. 4(b) shows that the precision degree has minimal impact on runtime, consistent with findings from prior research [44].
- The effect of the number of users: The running time of the PrivHFL algorithm is evaluated as shown in Fig. 5(a), where it is compared with existing methods, including HierFAVG [8], HFL-DP [34], and tMK-CKKS [18]. Although the running time of our algorithm is higher than that of the other two methods HierFAVG and HFL-DP, it remains within acceptable limits. HierFAVG [8] processes data in plaintext without offering any privacy guarantees, resulting in the fastest runtime. In contrast, HFL-DP [34] implements DP, which adversely affects the system's accuracy. Compared to [8,34], PrivHFL incurs a reasonable cost for privacy protection while preserving accuracy. Notably, tMK-CKKS [18] has the longest running time among all methods, primarily due to its requirement for two communication rounds in each iteration, as the edge must send encrypted aggregated model updates back to users for decryption. Consequently, the running time of tMK-CKKS increases significantly as the number of users rises.

As depicted in Fig. 5(a), the running time increases almost linearly with the number of users. This occurs because additional computations on encrypted elements are required as more users participate in the edge aggregation process. However, the growth

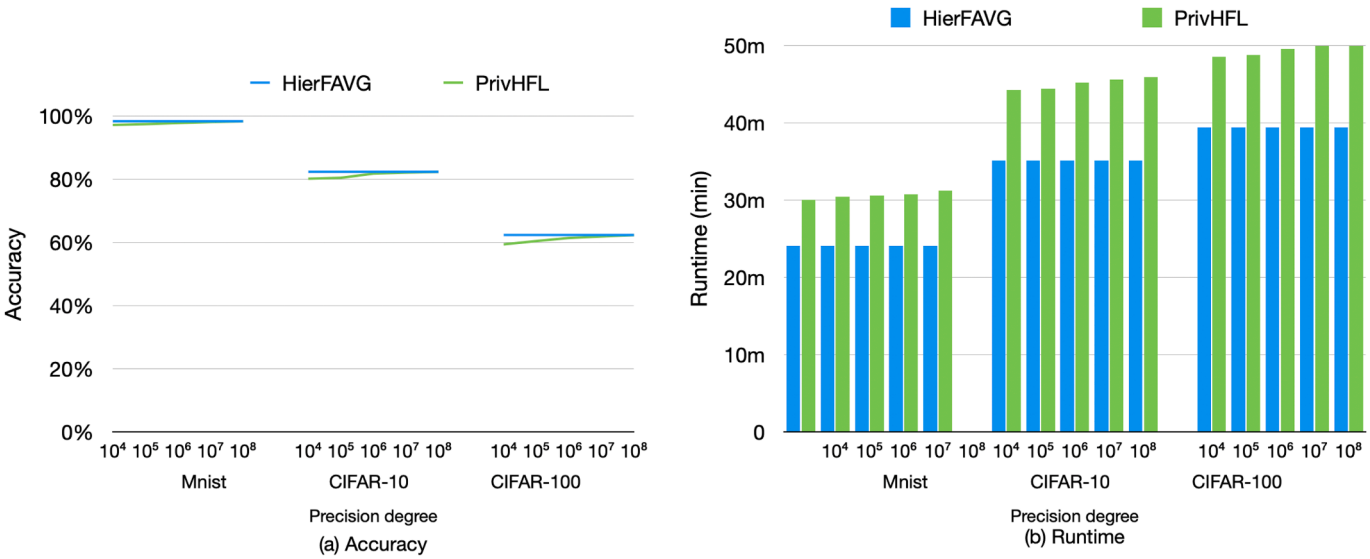


Fig. 4. Comparison of accuracy and runtime for PrivHFL for different precision degrees.

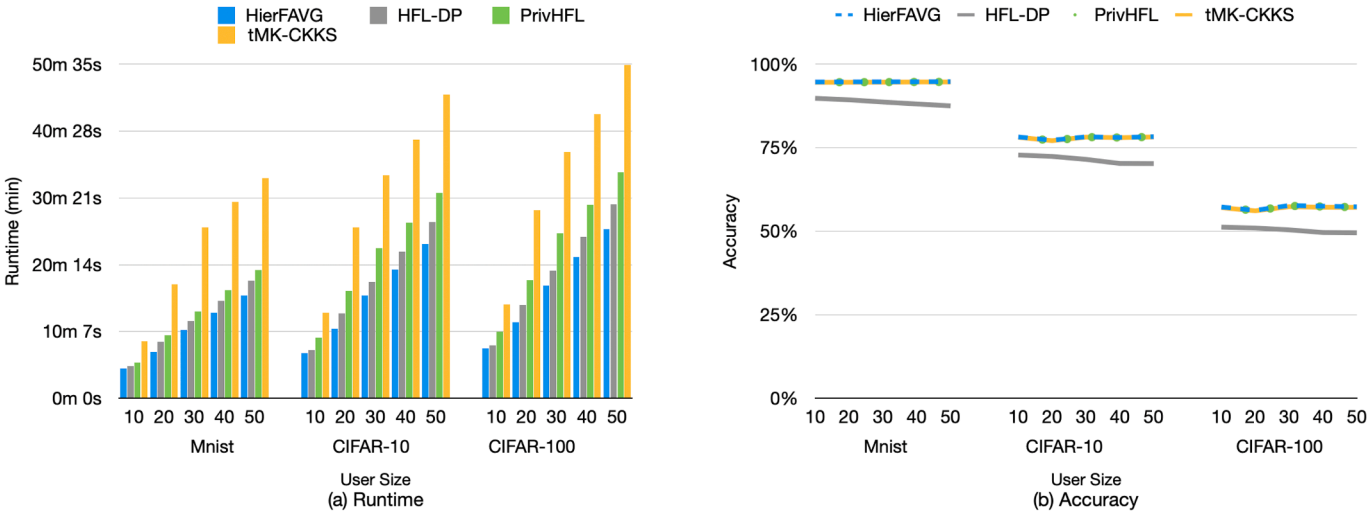


Fig. 5. Comparison of runtime and accuracy across four systems with different numbers of users.

Table 2
KGC setup time and total PrivHFL training time for different numbers of users on the MNIST dataset.

No. of Users	KGC Setup Time	PrivHFL Training Time
10	6 s	5 m 21 s
20	13 s	9 m 19 s
30	20 s	12 m 51 s
40	28 s	15 m 57 s
50	36 s	18 m 48 s

is predictable and acceptable due to its linearity. Meanwhile, HierFAVG, tMK-CKKS, and our scheme demonstrate minimal sensitivity to changes in the number of users in terms of accuracy. In contrast, HFL-DP experiences a decline in accuracy, as seen in Fig. 5(b).

- KGC initialization overhead: To quantify the initialization overhead introduced by the KGC, we measured the setup time separately from the training time. As shown in Table 2, the KGC setup time is consistently under 40 s even for 50 users, which is negligible compared to the total training time. Table 2

8. Conclusion and future work

This paper has presented PrivHFL, a hierarchical federated learning (HFL) approach designed for privacy preservation using threshold multi-key HE. Our solution aims to protect model parameters from curious users and servers. Experimental results show that our approach maintains high accuracy while effectively securing the model across IID and non-IID data settings. However, it is essential to note that using encryption increases the runtime of the HFL scheme, though it remains within acceptable limits. In future work, we aim to enhance the efficiency of PrivHFL and further optimize its overall runtime.

CRediT authorship contribution statement

Bayan Alzahrani: Writing – original draft, Software, Methodology, Formal analysis; **Dejun Yang:** Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] B. Mcmahan, Communication-Efficient Learning of Deep Networks from Decentralized Data, 2017, pp. 1273–1282.

[2] Q. Yang, Y. Liu, T. Chen, Y. Tong, Federated machine learning: concept and applications, ACM Transactions on Intelligent Systems and Technology (TIST), 10 (2019) 1–19. https://doi.org/10.1145/32989

[3] K. Bonawitz, et al., Towards federated learning at scale: system design, Proceedings of Machine Learning and Systems 1 (2019) 374–388.

[4] W. Li, Privacy-preserving federated brain tumour segmentation, in: Lecture Notes in Computer Science, 2019, pp. 133–141.

[5] A. Wainakh, A.S. Guinea, T. Grube, M. Mühlhäuser, Enhancing privacy via hierarchical federated learning, in: 2020 IEEE European Symposium on Security and Privacy Workshops, Genoa, Italy, 2020, pp. 344–347. https://doi.org/10.1109/EuroSPW51379.2020.00053

[6] M. Hao, H. Li, G. Xu, S. Liu, H. Yang, Towards efficient and privacy-preserving federated deep learning, in: IEEE International Conference on Communications (ICC), Shanghai, China, 2019, pp. 1–6.

[7] O. Aouedi, K. Piamrat, Towards a Scalable and Energy-Efficient Framework for Industrial Cloud-Edge-IoT Continuum, 2024.

[8] L. Liu, J. Zhang, S.H. Song, K.B. Letaief, ICC 2020 - 2020 IEEE International Conference on Communications (ICC), Technical Report, Cloud Hierarchical Federated Learning, 2020.

[9] B. Hitaj, G. Ateniese, F. Pérez-Cruz, Deep models under the GAN: information leakage from collaborative deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017. https://doi.org/10.1145/3133956.3134012

[10] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller, Inverting gradients - how easy is it to break privacy in federated learning?, Adv. Neural Inf. Process. Syst. 33 (2020), pp. 16937–16947.

[11] C. Ma, et al., On safeguarding privacy and security in the fram work of federated learning, IEEE Netw. 34 (4) (2020) 242–248. https://doi.org/10.1109/MNET.001.1900506

[12] R.L. Rivest, M.L. Dertouzos, On data banks and privacy homomorphisms, Found. Secure Comput. 4 (11) (1978) 169–180.

[13] C. Dwork, A firm foundation for private data analysis, Commun. ACM 54 (1) (2011) 86–95.

[14] A.C. Yao, Protocols for secure computations, in: 23rd Annual Symposium on Foundations of Computer Science, 1982, pp. 160–164.

[15] J. Chen, J. Xue, Y. Wang, L. Huang, T. Baker, Z. Zhou, Privacy-preserving and traceable federated learning for data sharing in industrial IoT applications, Expert Syst. Appl. 213 (2023) 119036. https://doi.org/10.1016/j.eswa.2022.119036

[16] B. Jayaraman, D. Evans, Evaluating differentially private machine learning in practice, in: 28th USENIX Security Symposium (USENIX Security 19, 2019, pp. 1895–1912.

[17] K. Bonawitz, et al., Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the ACM Conference on Computer and Communications Security, the ACM Conference on Computer and Communications Security, Association for Computing Machinery, 2017, pp. 1175–1191. https://doi.org/10.1145/3133956.3133982

[18] W. Du, M. Li, L. Wu, Y. Han, T. Zhou, X. Yang, A efficient and robust privacy-preserving framework for cross-device federated learning, Complex Intell. Syst. 9 (5) (2023) 4923–4937.

[19] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: International Conference on the Theory and Applications of Cryptographic Techniques, Springer-Verlag, 1999, pp. 223–238.

[20] J.H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: ASIACRYPT'17, Cham, Springer, 2017, pp. 409–437. https://doi.org/10.1007/978-3-319-70694-8-15

[21] L.T. Phong, Y. Aono, T. Hayashi, L. Wang, S. Moriai, Privacy-preserving deep learning via additively homomorphic encryption, IEEE Trans. Inf. Forensics Secur. 13 (5) (2018) 1333–1345. https://doi.org/10.1109/TIFS.2017.2787987

[22] J. Zhang, B. Chen, S. Yu, H. Deng, PEFL: a privacy-enhanced federated learning scheme for big data analytics, in: IEEE Globa Communications Conference (GLOBECOM), 2019, pp. 1–6.

[23] J. Dean, Large scale distributed deep networks, Adv. Neural Inf. Process. Syst. 25 (2012), pp. 1223–1231.

[24] Y. Rahulamathavan, Privacy-Preserving Similarity Calculation of Speaker Features using Fully Homomorphic Encryption, arXiv.org, 2022.

[25] H. Chen, W. Dai, M. Kim, Y. Song, Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019.

[26] J. Ma, S. Naas, S. Sigg, X. Lyu, Privacy-preserving federated learning based on multi-key homomorphic encryption, Int. J. Intell. Syst. 37 (9) (2022) 5880–5901.

[27] J. Bell, K. Bonawitz, A. Gascón, T. Lepoint, M. Raykova, Secure Single-Server Aggregation with (Poly)Logarithmic Overhead, 2020. https://doi.org/10.1145/3372297.3417885

[28] E. Sotthiwat, L. Zhen, Z. Li, C. Zhang, Partially encrypted multi-party computation federated learning, in: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Melbourne, Australia, 2021, pp. 828–835.

[29] R. Geyer, T. Klein, M. Nabi, Differentially Private Federated Learning: A Client Level Perspective, Technical Report, arXiv preprint, 2017.

[30] K. Wei, User-level privacy-preserving federated learning: analysis and performance optimization, IEEE Trans. Mob. Comput. 21 (9) (2022) 3388–3401.

[31] S. Truex, A hybrid approach to privacy-preserving federated learning, in: Proceedings of the ACM Conference on Computer and Communications Security, the ACM Conference on Computer and Communications Security, 2019, pp. 1–11.

[32] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, H. Ludwig, Hybridalpha: an efficient approach for privacy-preserving federated learning, in: Proceedings of the ACM Conference on Computer and Communications Security, the ACM Conference on Computer and Communications Security, Association for Computing Machinery, 2019, pp. 13–23.

[33] W.Y.B. Lim, Decentralized edge intelligence: a dynamic resource allocation framework for hierarchical federated learning, IEEE Trans. Parallel Distrib. Syst. 33 (2022) 536–550.

[34] L. Shi, J. Shu, W. Zhang, Y. Liu, HFL-DP: hierarchical federated learning with differential privacy, in: 2021 IEEE Global Communications Conference (GLOBECOM), Madrid, Spain, 2021, pp. 1–7.

[35] G. Bao, P. Guo, Federated learning in cloud-edge collaborative architecture: key technologies, applications and challenges, J. Cloud Comput. 11 (1) (2022), pp. 94.

[36] V. Lyubashevsky, C. Peikert, O. Regev, On Ideal Lattices and Learning with Errors over Rings, 6110, Springer, Berlin, Heidelberg, Berlin, Heidelberg, 2010.

[37] A. Shamir, How to share a secret, Commun. ACM 22 (11) (1979) 612–613.

[38] R. Shokri, V. Shmatikov, Privacy-preserving deep learning, in: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, the 22nd ACM SIGSAC Conference on Computer and Communications Security, 2015. https://doi.org/10.1145/2810103.2813687

[39] C. Hahn, H. Kim, M. Kim, J. Hur, VerSA: verifiable secure aggregation for cross-device federated learning, IEEE Trans. Dependable Secure Comput. 20 (1) (2023) 36–52.

[40] E. B. B. L. C. Simon Allen L. Roginsky, R. D. Scott, Recommendation for Pair-Wise Key Establishment Using Integer Factorization Cryptography, Technical Report NIST SP 800-56Br2, National Institute of Standards and Technology, Gaithersburg, MD, 2019.

[41] M. Albrecht, Homomorphic Encryption Standard, Technical Report, Tech, org, Toronto, Canada, 2018. Rep 11.

[42] L. Yann, The MNIST Database of Handwritten Digits, 1998. http://yann.lecun.com/exdb/mnist/.

[43] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, 2009.

[44] W. Jin, FEDML-HE: an efficient homomorphic-encryption-based privacy-preserving federated learning system, in: International Workshop on Federated Learning in the Age of Foundation Models in Conjunction With NeurIPS, 2023.